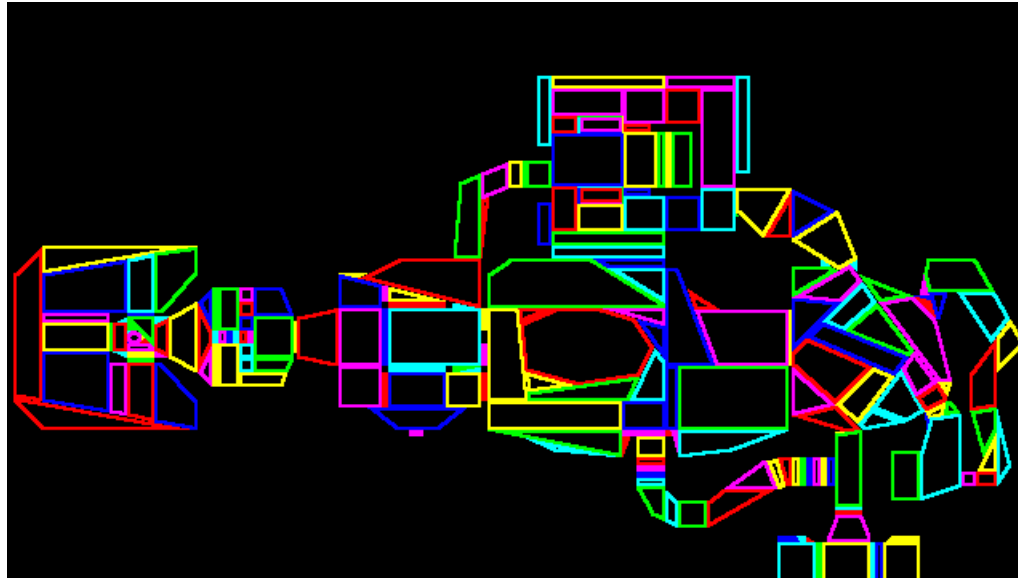


# Bounding Volume Hierarchies



Some Slides/Images adapted from Marschner and Shirley and David Levin

# Agenda

- Motivation: Common Geometric Queries in Graphics
- Bounding Volumes
  - Spheres
  - Boxes (AABB, OOBB)
- Constructing Object-Partitioning Hierarchies
  - Sphere Trees
  - AABB Trees
- Space-Partitioning Hierarchies
  - Uniform Spatial Subdivision
  - Axis-Aligned Spatial Subdivision

# Geometric modeling and geometric queries

Closest point on a triangle?

Project  $p$  to  $p'$  in plane of triangle.

Calculate barycentric coordinates of  $p'$ .

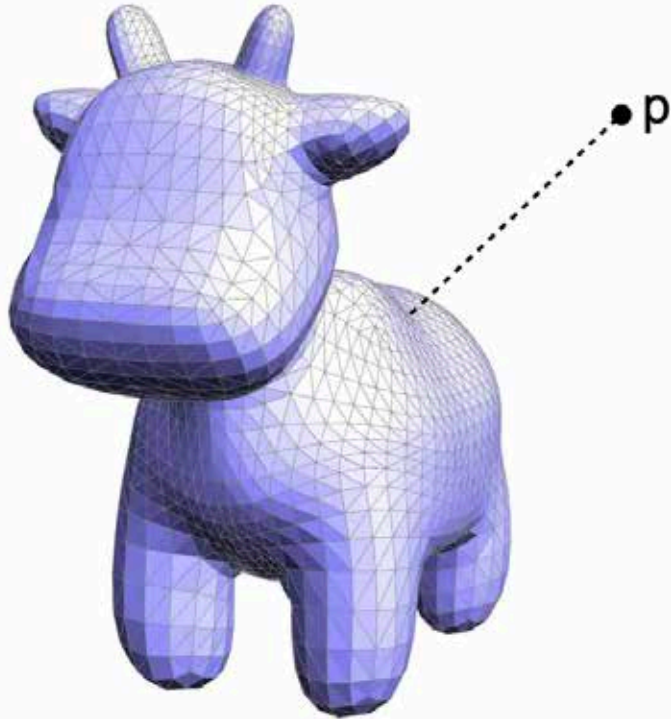
Clamp coordinates to  $[0,1]$ .

Point with clamped coordinates is closest to  $p$ .

Loop over all triangles and keep the closest.

What is the complexity?

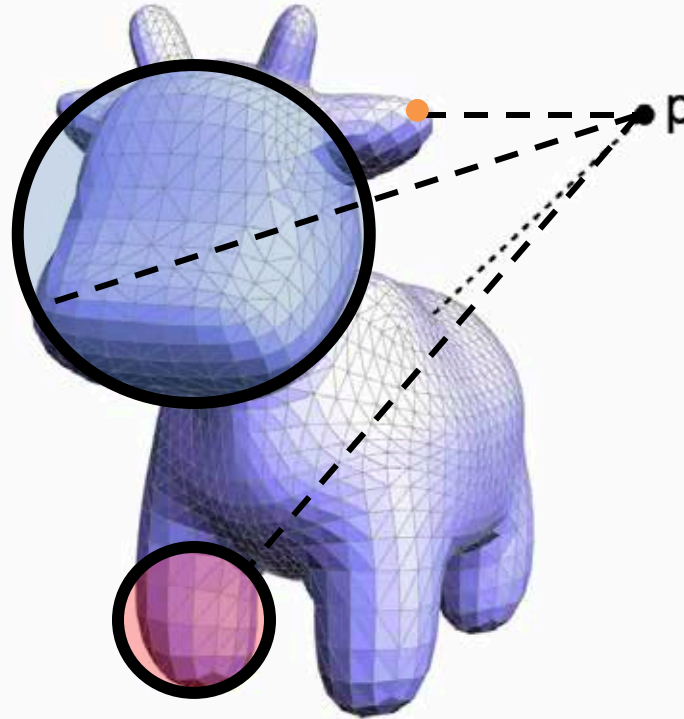
What if the object has a billion triangles?



What point on the mesh is closest to  $p$ ?

# Geometric modeling and geometric queries

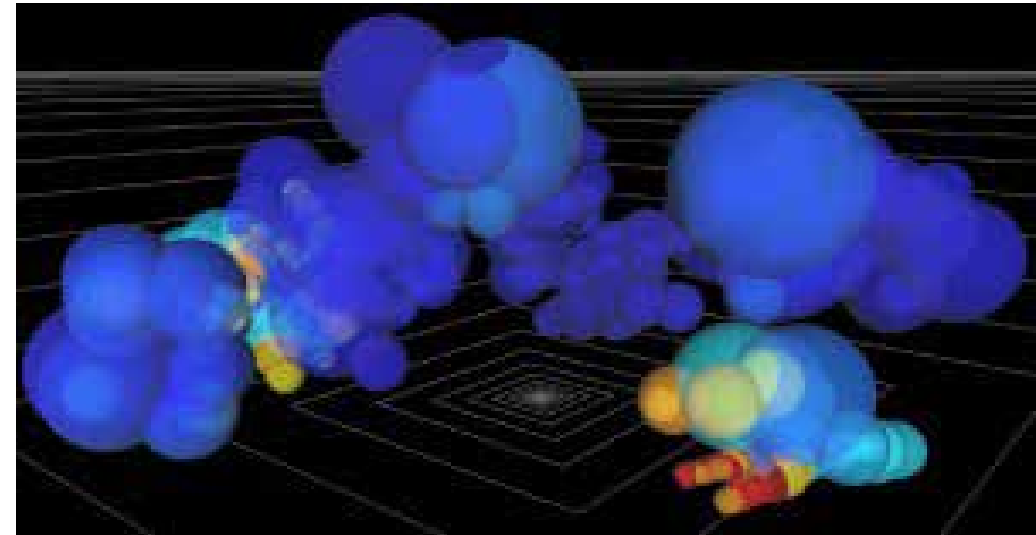
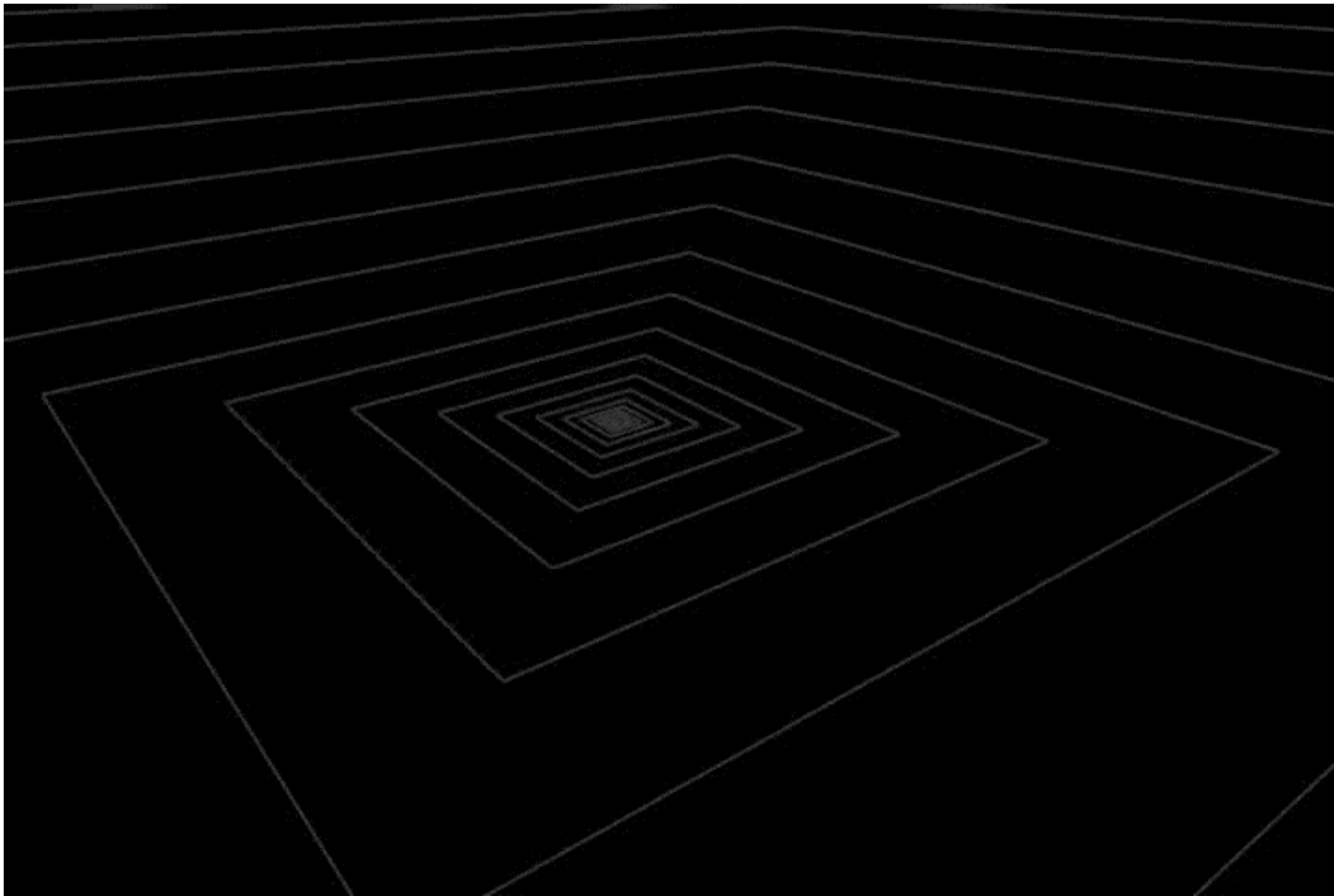
- Closest/furthest point to a sphere with center  $c$  and radius  $r$  is:  
 $c \pm r * \text{normalized}(p-c)$
- Closest point on red sphere is further than furthest point on blue sphere.
- Closest point on blue sphere is further than the orange point on a triangle.



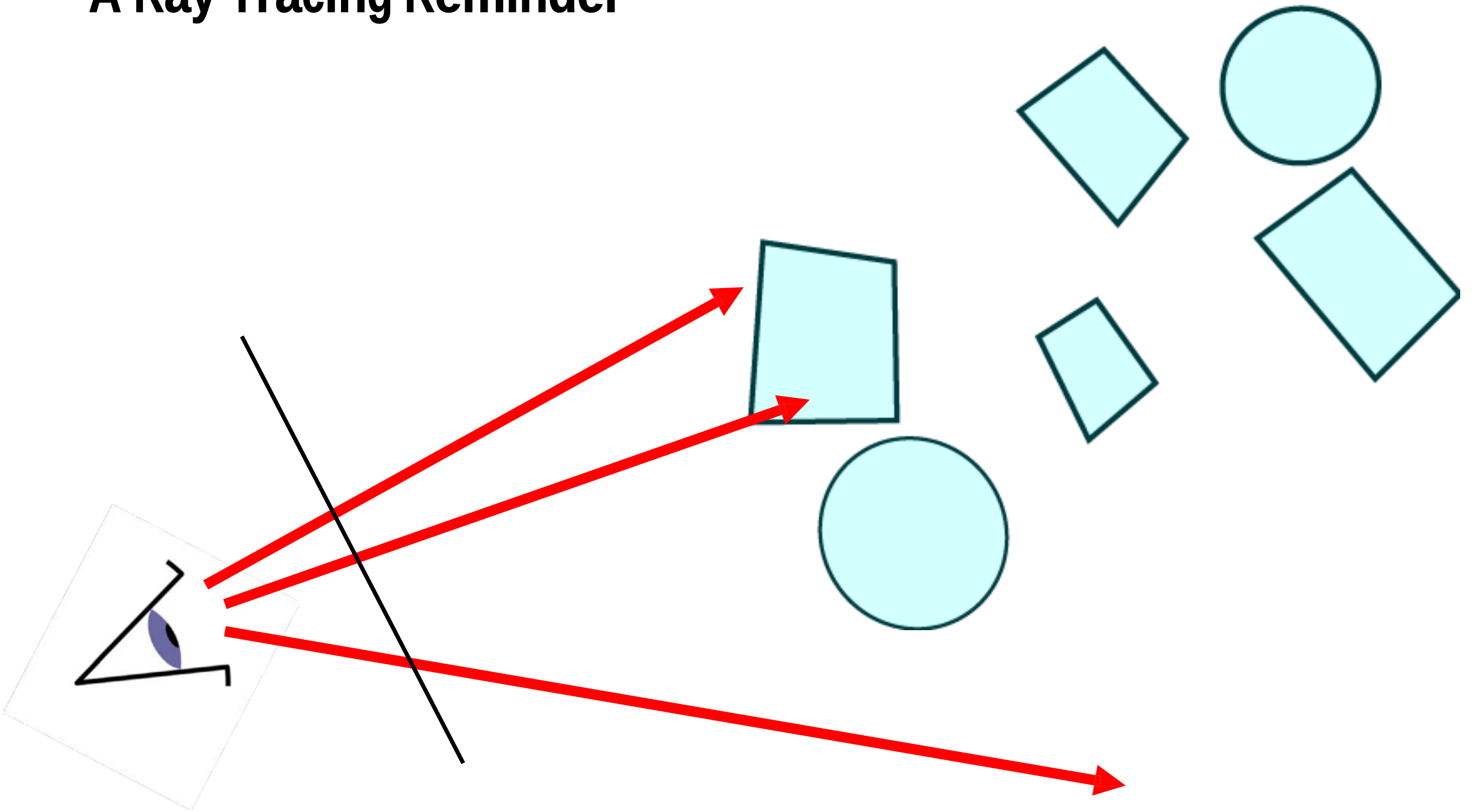
What point on the mesh is closest to  $p$ ?

# Collision Intersection

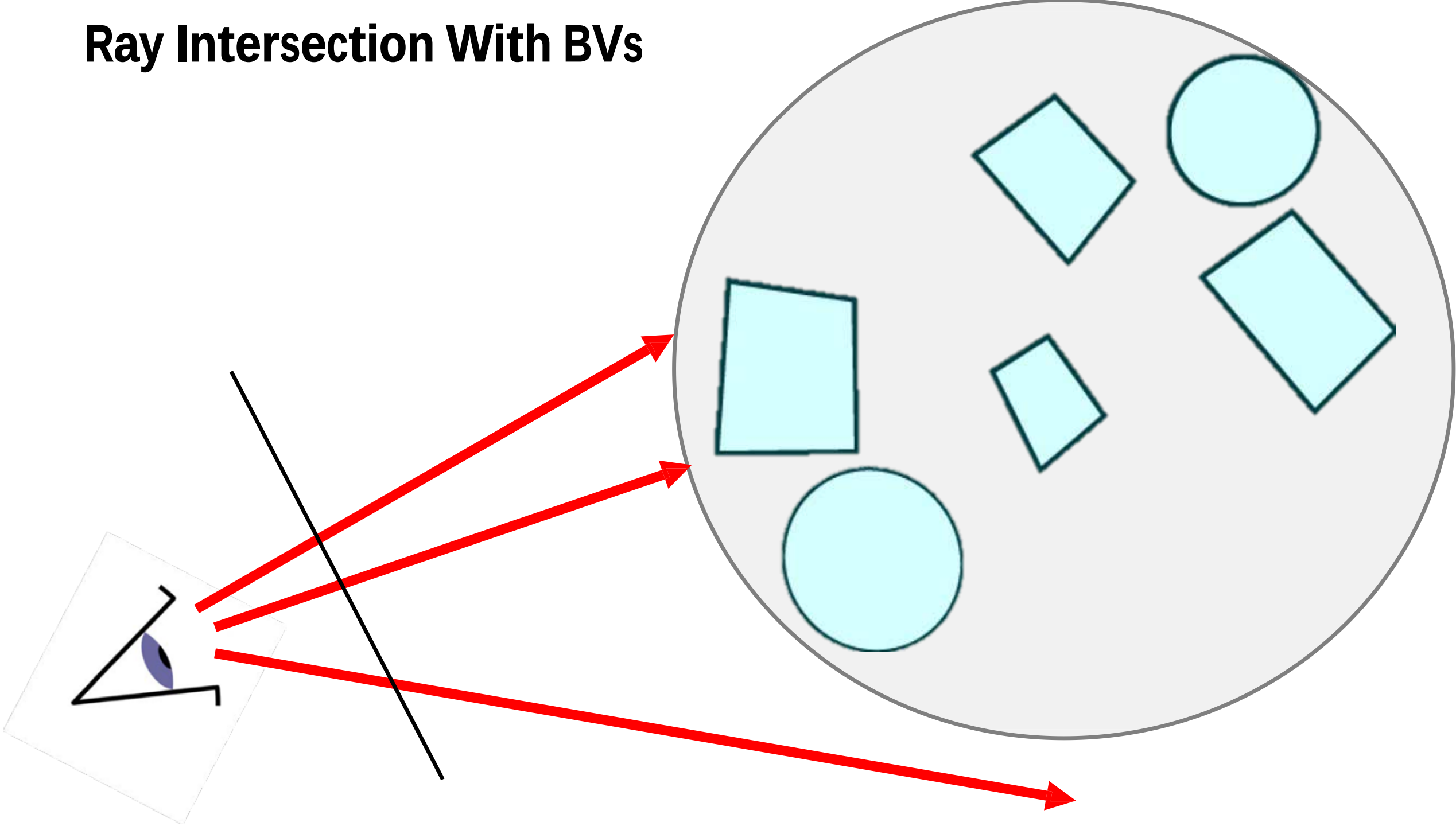
How can you test if two spheres intersect?



# A Ray Tracing Reminder



# Ray Intersection With BVs



# Bounding volumes

Quick way to avoid expensive testing intersections and collisions:

    bound object with a simple volume **bvol** (volume encloses object)

If a ray/object doesn't hit/intersect **bvol**,

    it doesn't hit/intersect the object

else

    test object for hit/intersect

Cost: more for hits and near misses, less for far misses

Worth doing? Yes if:

    Cost of **bvol** intersection test is small: simple shapes (spheres, boxes, ...)

    Cost of object intersect test is large: **bvol** most useful for complex objects

    Tightness of fit is good:

        Loose fit leads to extra object intersections

        Tradeoff between tightness and **bvol** intersection cost

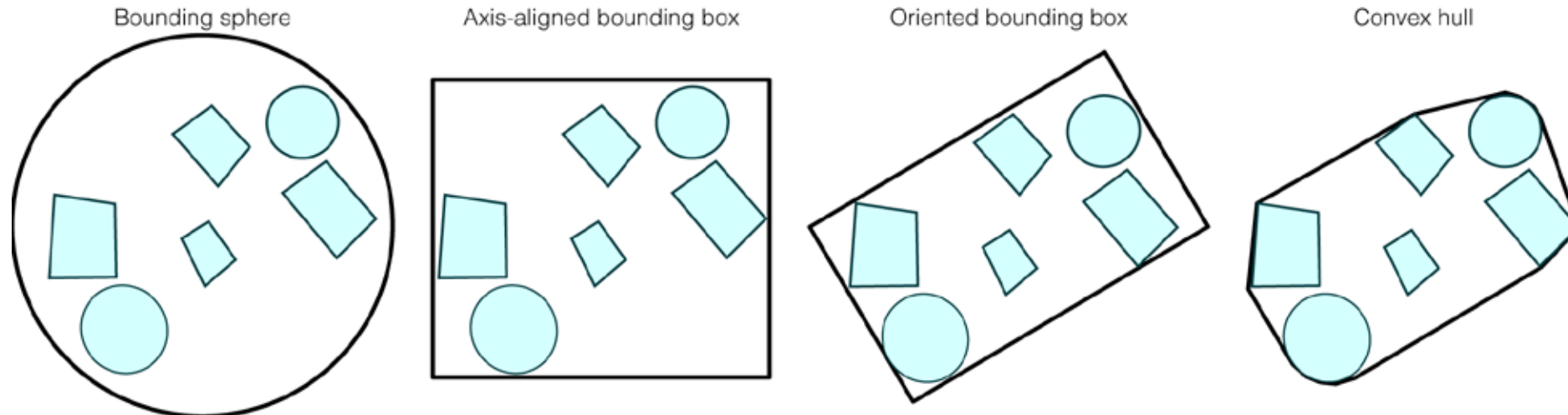
# Choice of bounding volumes

Spheres: easy to intersect, not always tight.

Axis-aligned Bounding Boxes (AABBs): easy to intersect, tighter for axis-aligned objects.

Oriented bounding boxes (OBBs): easy to intersect (transformation cost),  
tighter than AABBs.

Convex Hull: not as easy to intersect as the above, tighter than the above.

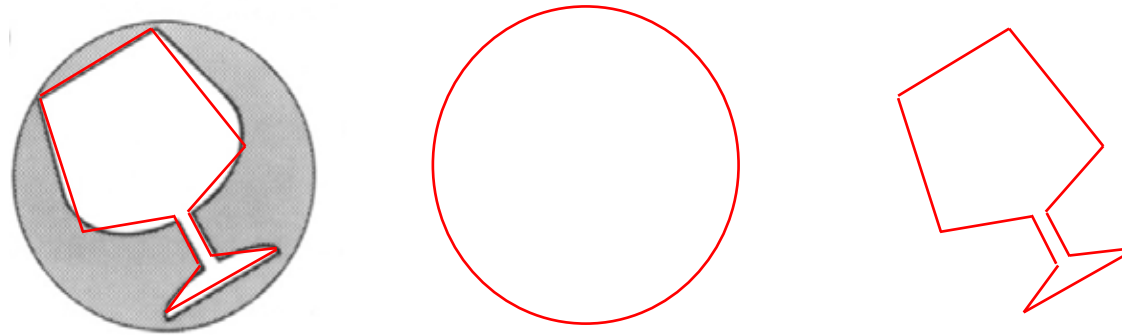


# Proxy Geometry vs. Bounding volumes

Another concept often used in CG is proxy geometry or Level-Of-Detail LOD.

A proxy is a simplified representation of the object, that can be used as the object when rendering and processing speed is more important than visual accuracy.

Note, that proxy geometry is an approximation to the object and typically not a bounding volume. And a bounding volume itself is typically not a good visual proxy for an object.

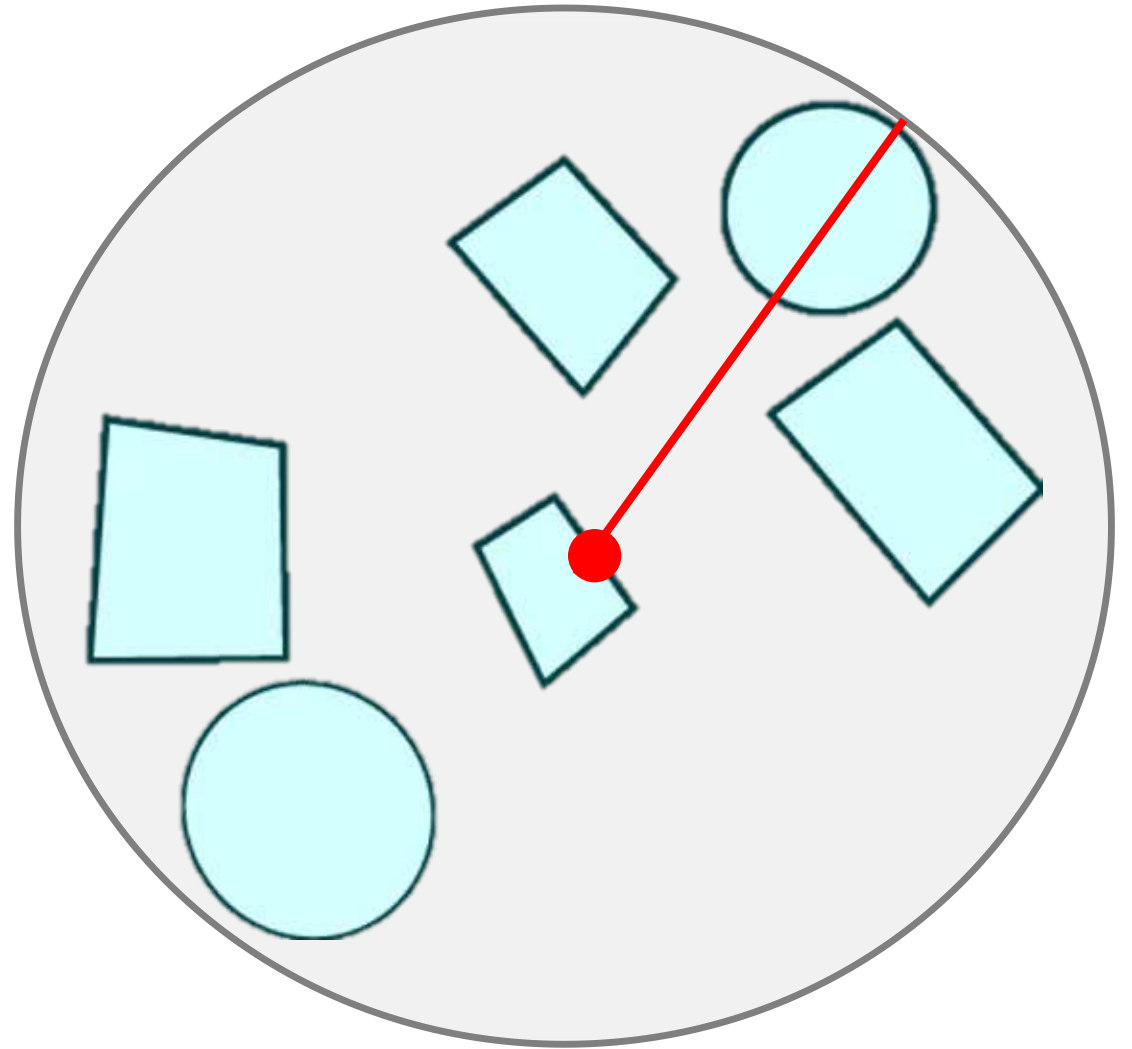


# Building a Bounding Sphere

Parameters of a Sphere:

1. Center =  $\mathbf{c} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}^i$
2. Radius =  $r = \max(\|\mathbf{v}^i - \mathbf{c}\|)$

$\mathbf{v}^i \in \text{Vertices}$



# Ray-Sphere Intersection

Substitute ray equation into implicit equation for sphere

$$(\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) \cdot (\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) - r^2 = 0$$

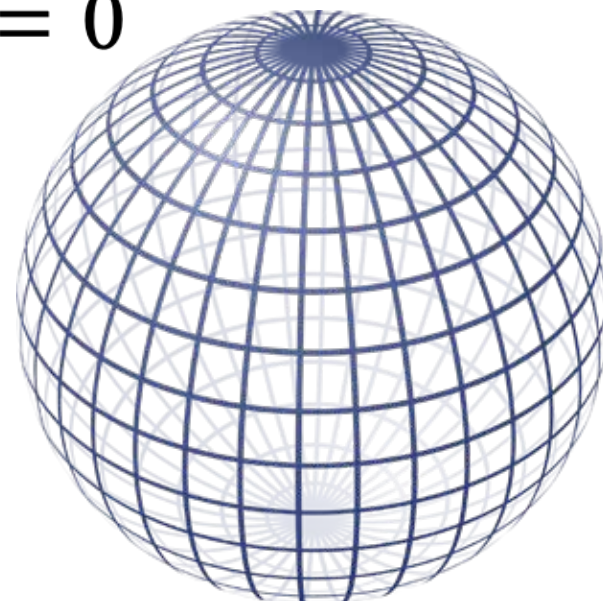
Rearrange

$$(\vec{\mathbf{d}} \cdot \vec{\mathbf{d}})t^2 + 2\vec{\mathbf{d}} \cdot (\mathbf{e} - \mathbf{c})t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - r^2 = 0$$

Looks familiar...

$$At^2 + Bt + C = 0$$

It's a quadratic! (can use the quadratic equation)



# Axis aligned bounding boxes

Probably easiest to implement

Computing for primitives

Cube: duh!

Sphere, cylinder, etc.: pretty obvious

Groups or meshes: min/max of component parts

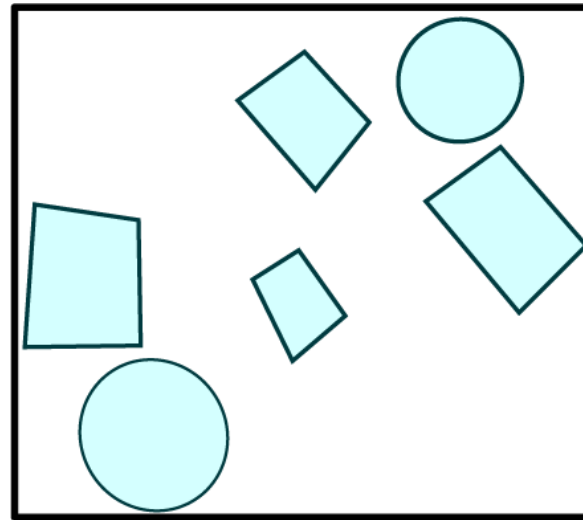
$$x_{\min} = \min(v_x^i)$$

$$x_{\max} = \max(v_x^i)$$

$$y_{\min} = \min(v_y^i)$$

$$y_{\max} = \max(v_y^i)$$

$$\mathbf{v}^i \in \text{Vertices}$$



# Ray-AABB Intersection

How to intersect an AABB with a ray  $\mathbf{p}(t) = \mathbf{p}_e + \mathbf{p}_d * t$

Treat them as an intersection of slabs (see book 12.3.1)

$$t_{x\min} = (x_{\min} - x_e) / x_d$$

$$t_{x\max} = (x_{\max} - x_e) / x_d$$

$$t_{y\min} = (y_{\min} - y_e) / y_d$$

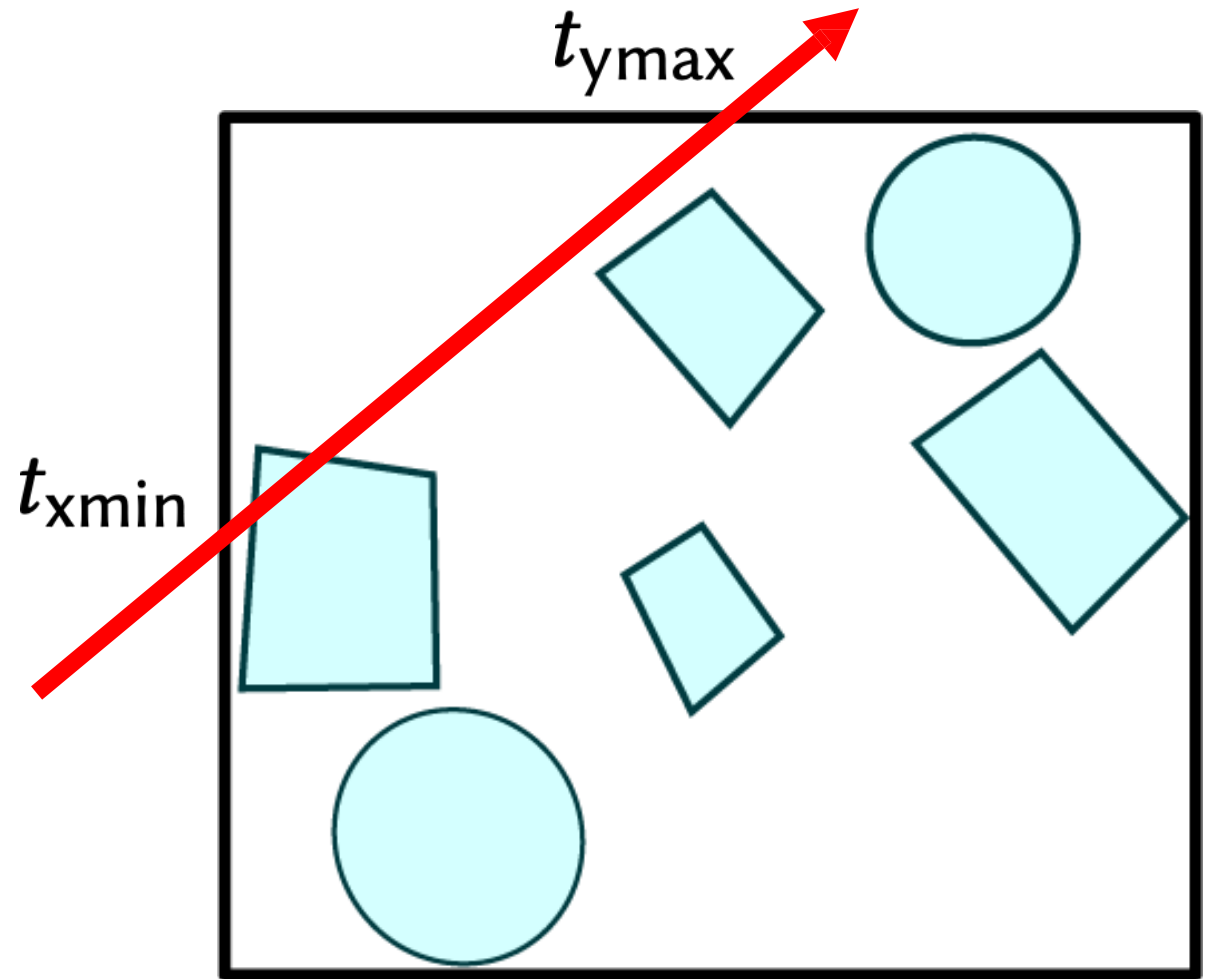
$$t_{y\max} = (y_{\max} - y_e) / y_d$$

# Ray-AABB Intersection

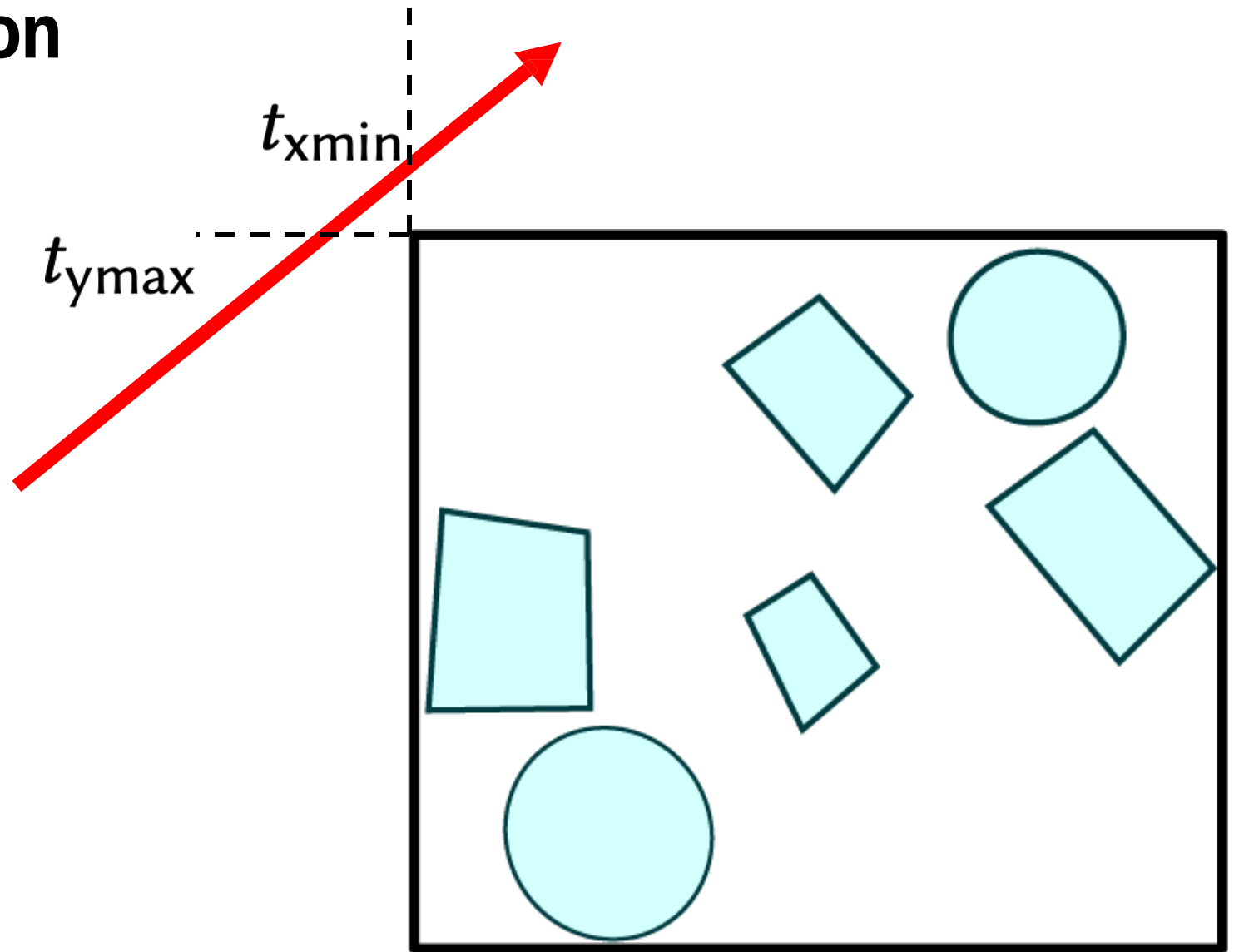
$$t_{xmin} < t_{ymax}$$

$t_{xmin}$

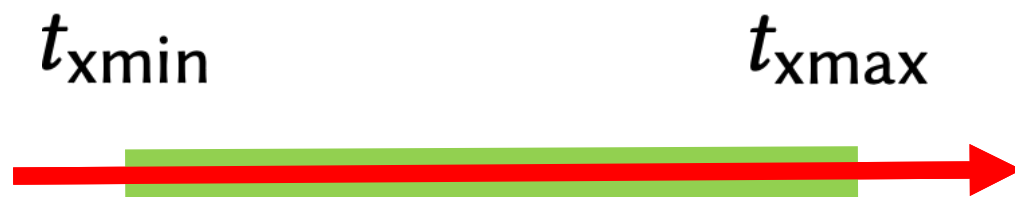
$t_{ymax}$



# Ray-AABB Intersection



# Ray-AABB Intersection

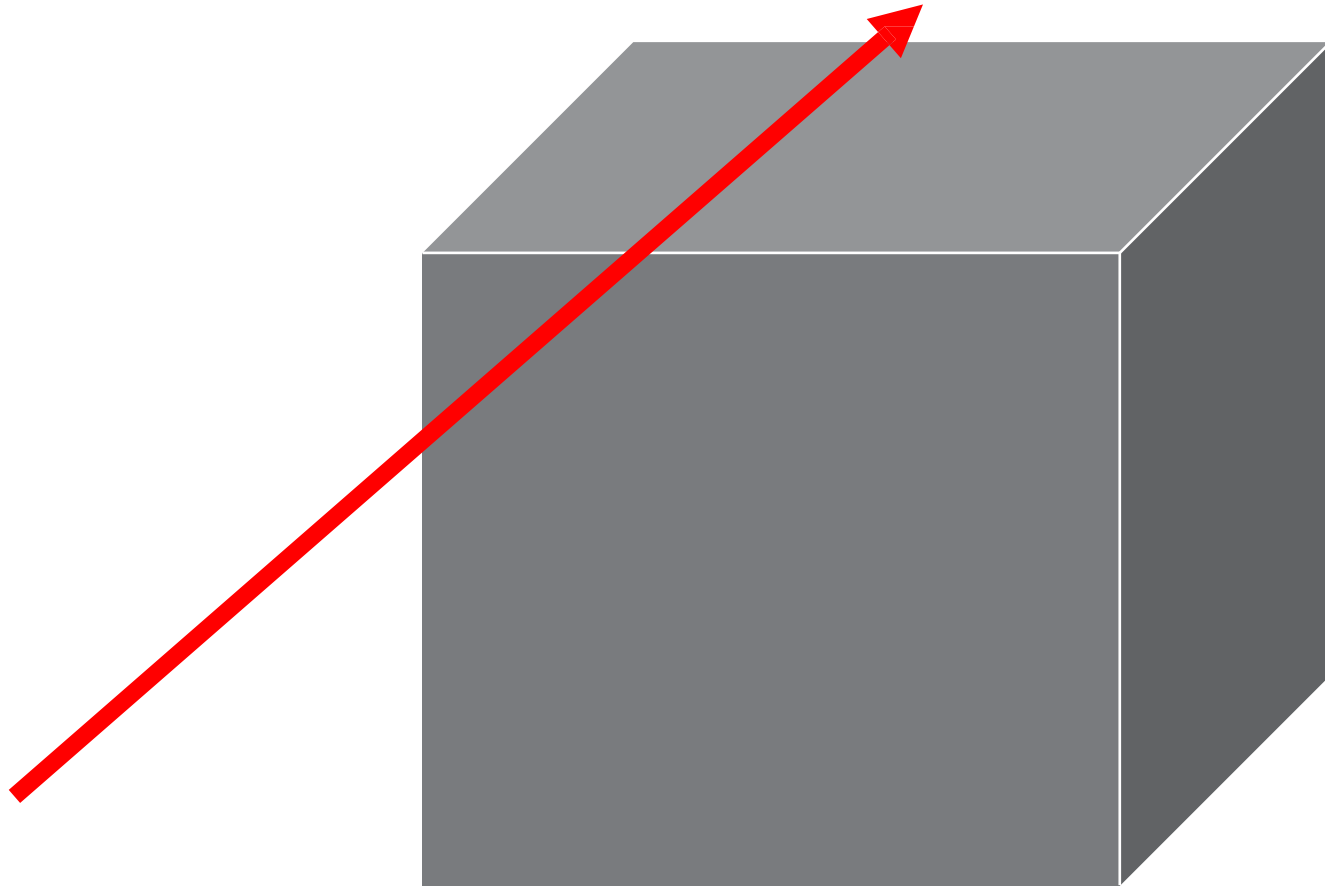


Intersection of Intervals ?

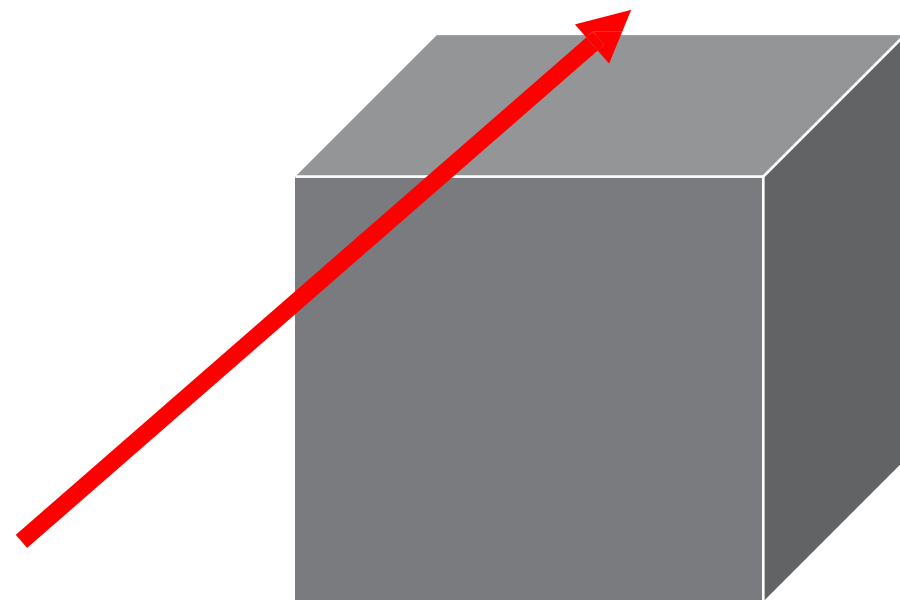
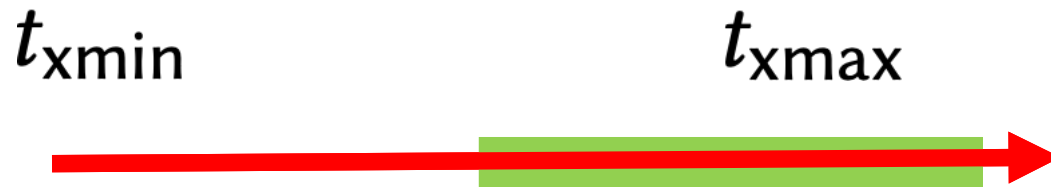


$$\max(t_{xmin}, t_{ymin}) < \min(t_{xmax}, t_{ymax})$$

# What happens in 3D ?

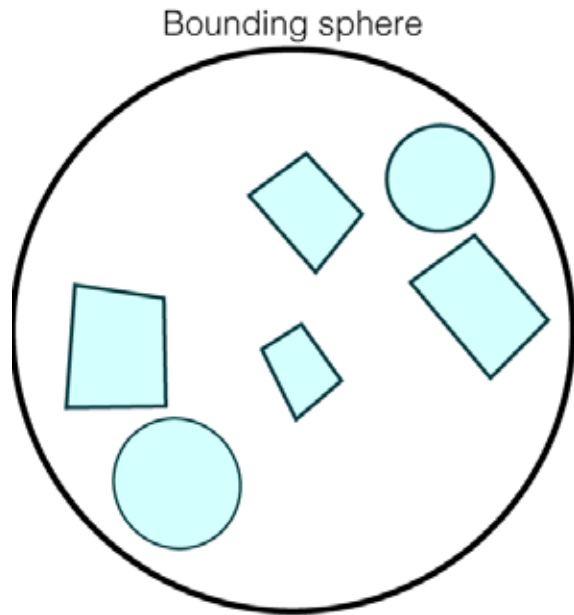


# Ray-AABB Intersection

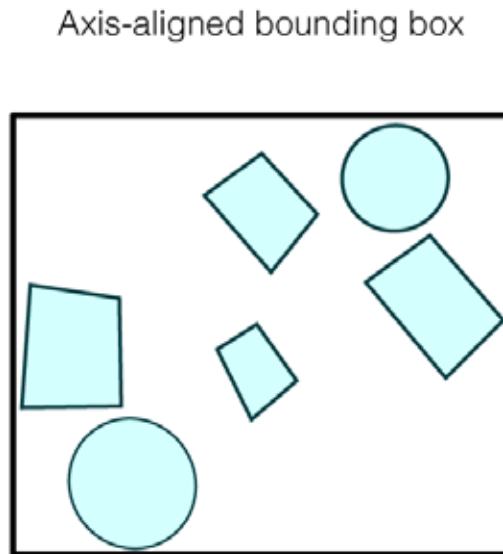


# Bounding Volumes (BVs)

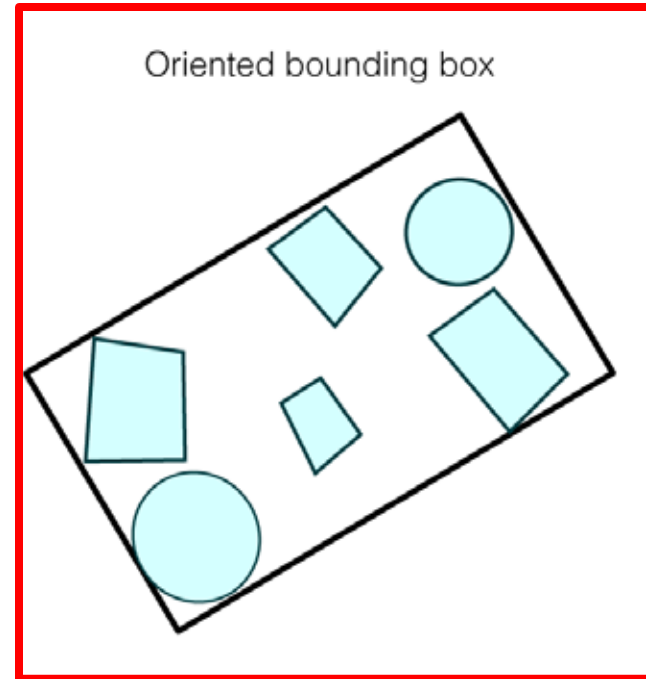
“Simple” geometry that fully encloses a **collection** of other geometry



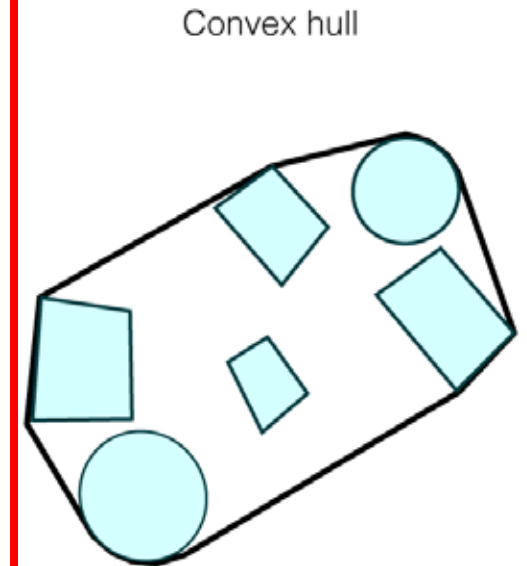
Sphere  
(a lot)



AABB  
(a lot)



OBB  
(a little)

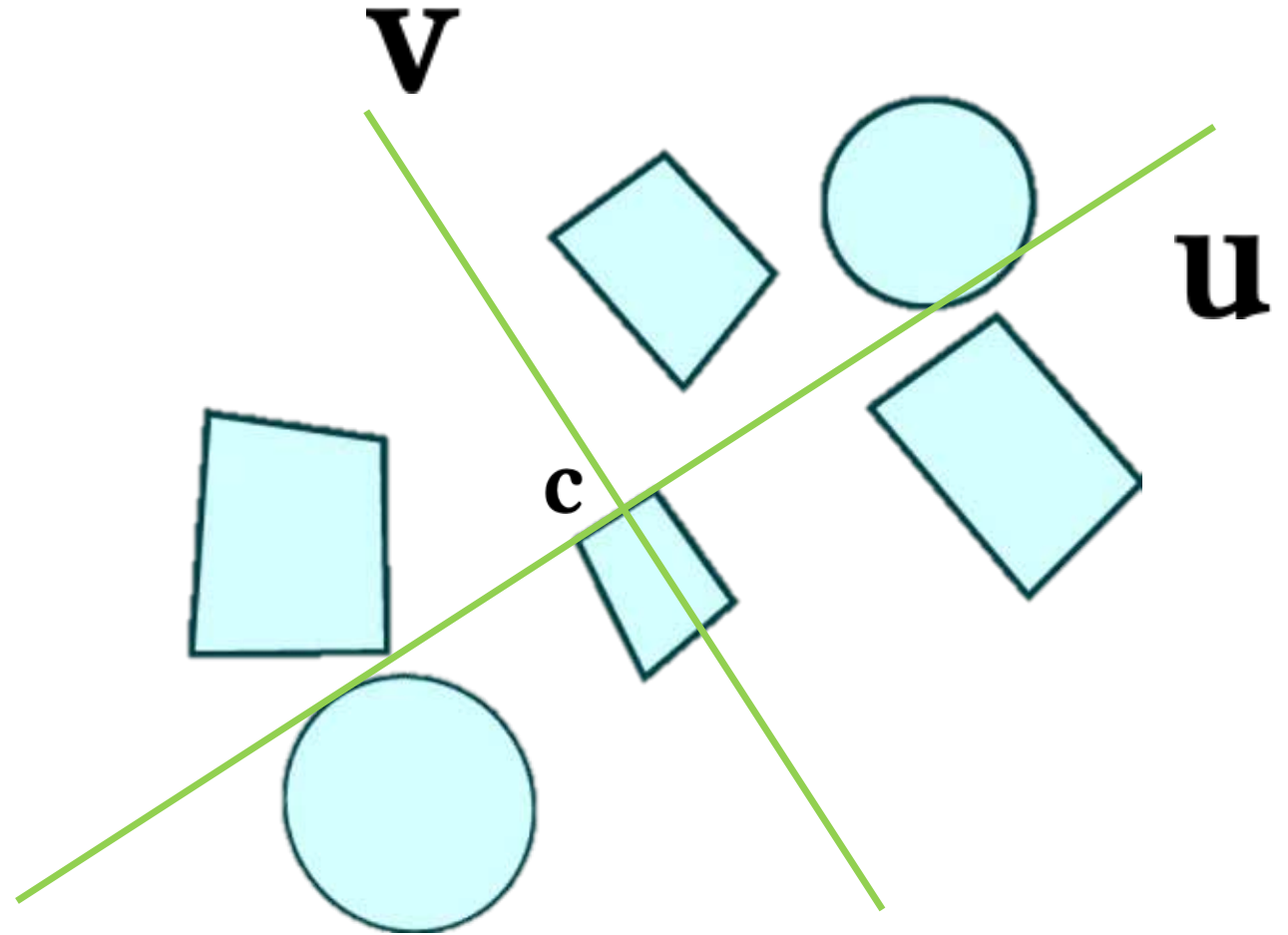


Convex Hull  
(nope!)

# Building an Object-Oriented Bounding Box (OOBB)

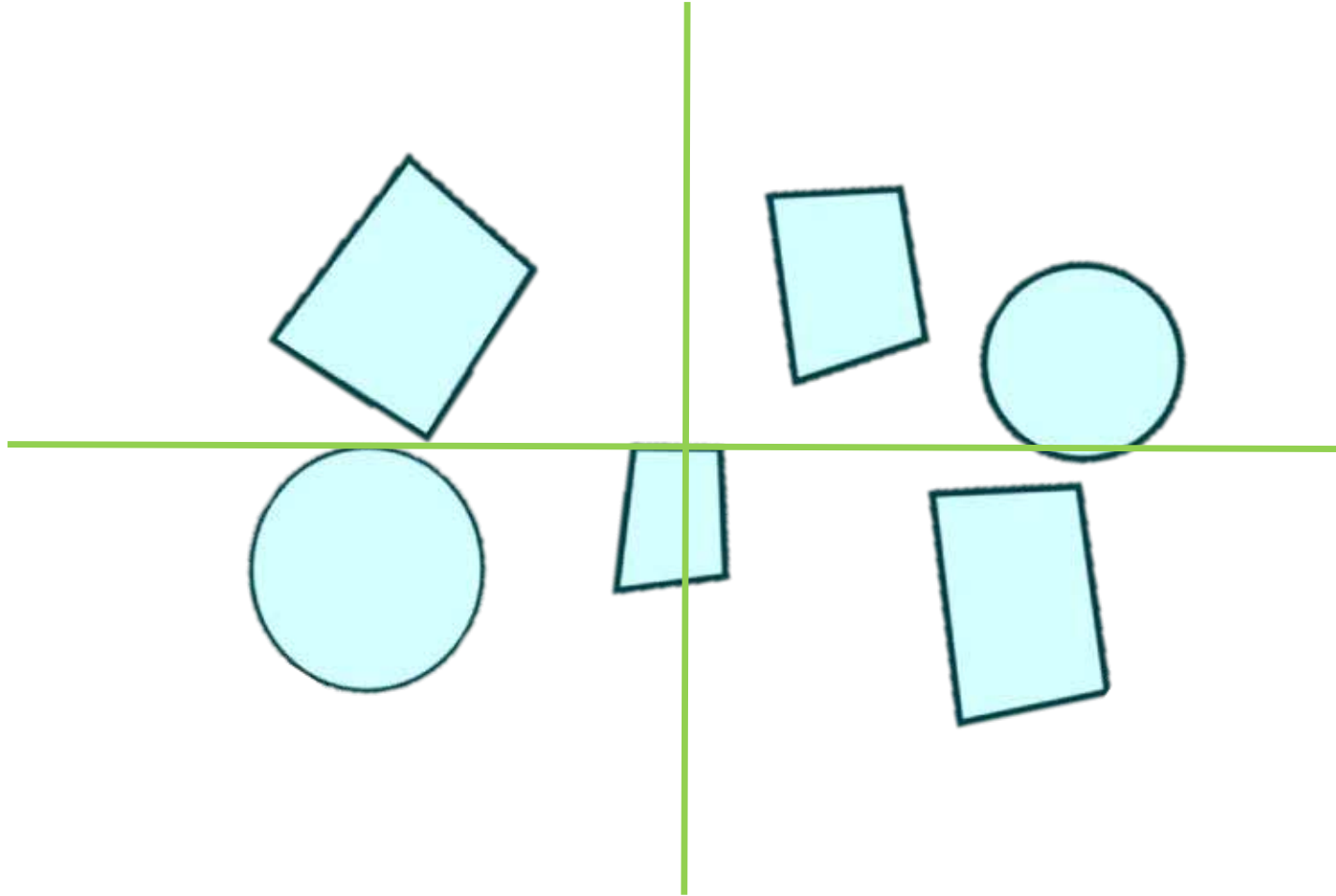
$$\mathbf{c} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}^i$$

$$[\mathbf{u} \quad \mathbf{v}]$$



Find directions of maximum and minimum variance

# Building an Object-Oriented Bounding Box (OOBB)



Build Rotation Matrix

# Implementing a bounding volume

Add new Surface subclass, *BoundedSurface*

Contains a *bvol* and a reference to a *surface*

Intersection method:

    if (!*bvol.intersect(ray,t)*) :

        return false

    else :

        return *surface.intersect(ray,t)*

This change is transparent to the renderer (only it might run faster).

# Implementing a bounding volume hierarchy

A *BoundedSurface* can contain a *surface* list.

Any *surface* in this list might also be a *BoundedSurface*  
=> A bounding volume hierarchy

# Spatial Data Structures

Basic Idea – asymptotic improvement in spatial queries by subdividing

Two types of subdivisions – ***object-based*** and *spatial* Our

*object-based data structures will be boundary volume hierarchies or BVHs.*

*BVHs are hierarchies of BVs represented by trees*

# Assignment #3 Tasks

| File                                                        | What it does                                                                       |
|-------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>insert_box_into_box.py</code>                         | Grow box <code>B</code> so it also encloses box <code>A</code> .                   |
| <code>insert_triangle_into_box.py</code>                    | Grow a box so it encloses a triangle's 3 corners.                                  |
| <code>ray_intersect_box.py</code>                           | Slab test: does a ray hit a <i>solid</i> AABB within <code>[min_t, max_t]</code> ? |
| <code>ray_intersect_triangle.py</code>                      | Marschner–Shirley ray–triangle intersection.                                       |
| <code>ray_intersect_triangle_mesh_brute_force.py</code>     | <code>O(n)</code> closest ray–mesh hit (reference).                                |
| <code>AABBTree.py</code>                                    | Build the hierarchy: split at the midpoint of the longest axis.                    |
| <code>AABBTree_ray_intersect.py</code>                      | Recursive, box-pruned closest ray–object query.                                    |
| <code>nearest_neighbor_brute_force.py</code>                | <code>O(n)</code> nearest point to a query (reference).                            |
| <code>point_box_squared_distance.py</code>                  | Squared distance from a point to a solid box.                                      |
| <code>point_AABBTree_squared_distance.py</code>             | Priority-queue nearest-neighbour query (not recursive!).                           |
| <code>box_box_intersect.py</code>                           | Do two AABBs overlap? (separating-axis test)                                       |
| <code>find_all_intersecting_pairs_using_AABBTrees.py</code> | Broad phase: overlapping leaf-box pairs between two trees.                         |

# AABB Tree Construction

Make AABB for whole scene/object, then split into two parts

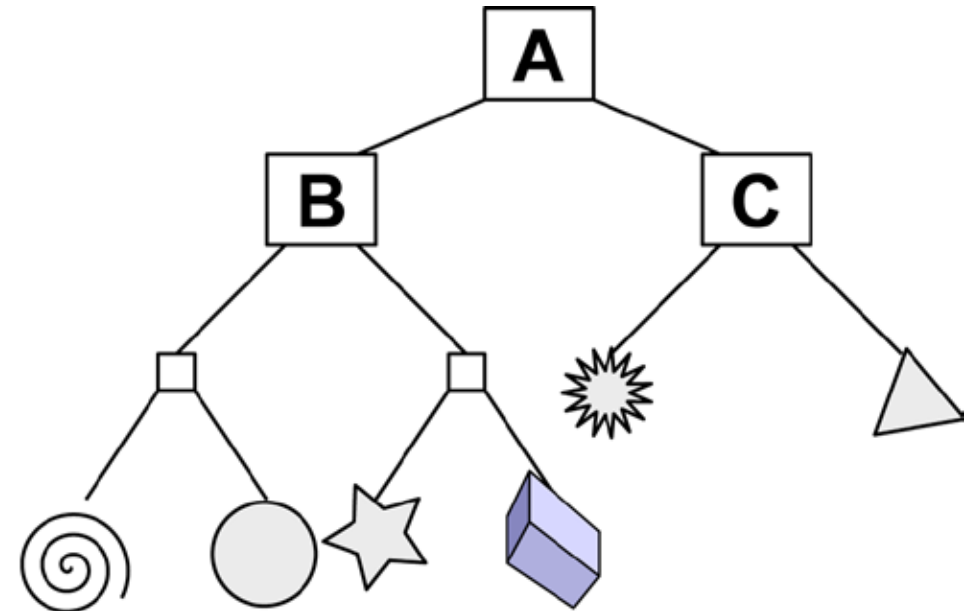
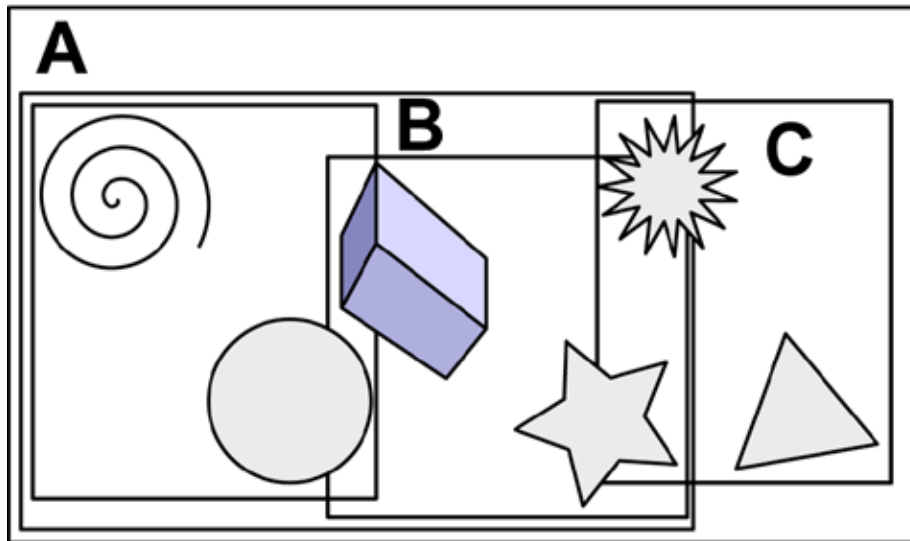
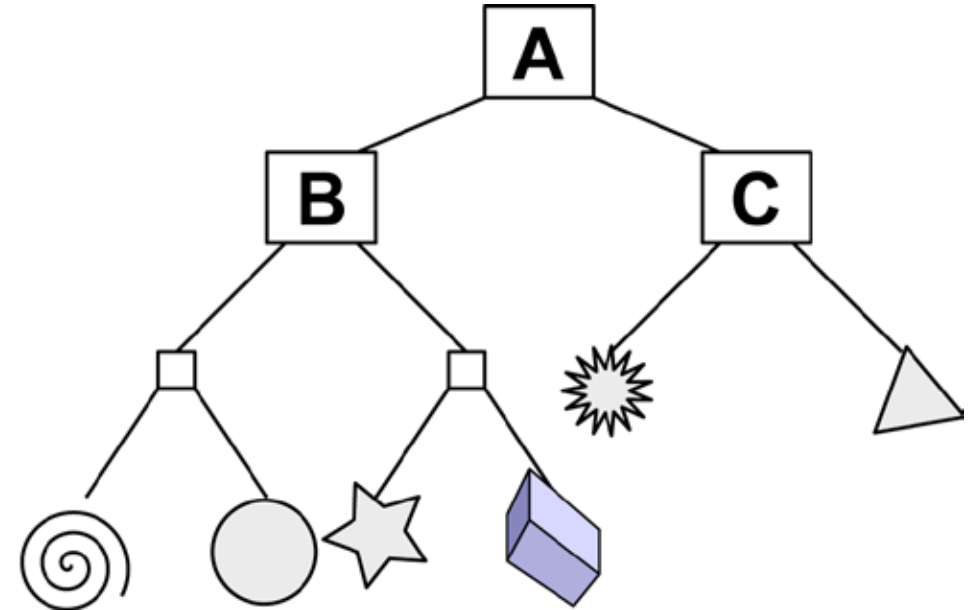
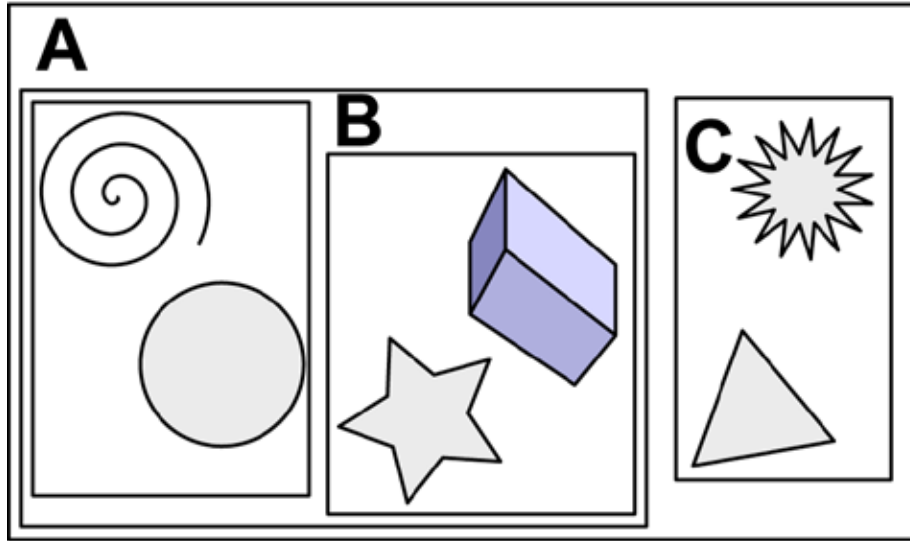
- Recurse on parts.
- Stop when there is one (or a few) object/triangle in your box.

How to split into parts?

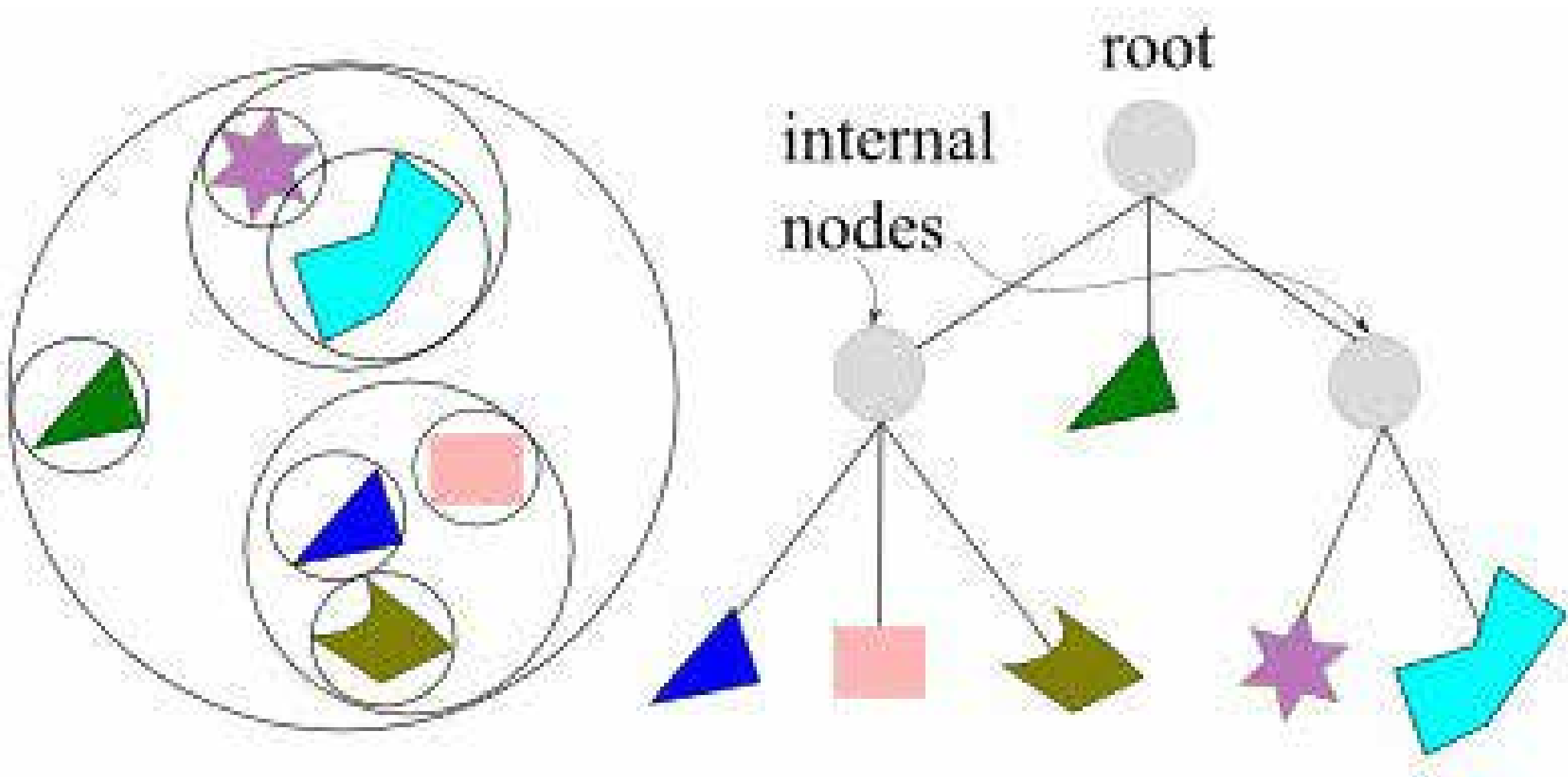
Space based: partition objects based on value relative to the center of longest dimension.

Object based: sort the objects along the longest dimension and divide them equally.

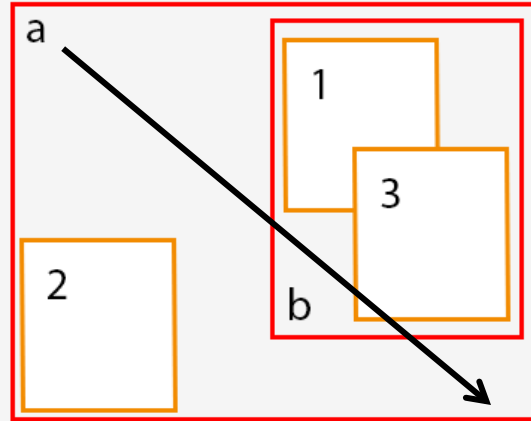
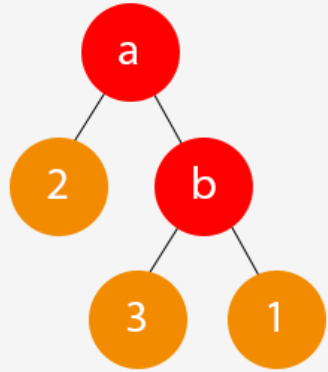
# AABB Tree Construction (object based)



# Sphere Trees



# Ray and AABB tree Intersection



DFS traversal of tree nodes!

```
def bvh.intersect(ray,t):  
    if (aabb== null || !aabb.intersect(ray,t)):  
        return false  
    else:  
        i1=left.intersect(ray,t1)  
        i2=right.intersect(ray,t2)  
        if (i1 && i2) :  
            t=min(t1,t2)  
            return true  
        if (i1) :  
            t=t1  
            return true  
        if (i2) :  
            t=t2  
            return true  
        return false
```

# BVH Distance Queries

```
def minDistance(bvNode, point, currentMin) :  
    if (isLeaf(bvNode)) :  
        d1=d2=minDist(bvNode.object, point)  
    else :  
        d1=minDistance(bvNode.left, point, currentMin)  
        d2=minDistance(bvNode.right, point, currentMin)  
    if (min(d1,d2) > currentMin) :  
        return currentMin  
    return min(d1,d2)
```

Is DFS traversal of tree nodes efficient?  
BFS with a priority queue!

# BVH Intersection Queries

```
leaf_pairs ← {}  
if (root_A.box Ç root_B.box) :  
    Q.insert(root_A, root_B )  
while Q not empty :  
    {nodeA,nodeB} ← Q.pop  
    if (nodeA and nodeB are leaves) :  
        leaf_pairs.insert( node_A, node_B )  
    elif (node_A is a leaf) : # symmetrically for node_B  
        if (node_A.box Ç node_B.left.box) :  
            Q.insert( node_A, node_B.left ); # symmetrically for node_B.right  
        else :  
            if (node_A.left.box Ç node_B.box) :  
                Q.insert( node_A.left, node_B )  
            if (node_A.right.box Ç node_B.box) :  
                Q.insert( node_A.right, node_B )  
            if (node_A.box Ç node_B.left.box) :  
                Q.insert( node_A, node_B .left )  
            if (node_A.box Ç node_B.right.box) :  
                Q.insert( node_A.left, node_B.right )
```

# Triangle-Triangle intersection

$T_1$  intersects  $T_2 \iff$  at least one tri edge intersects the other tri.

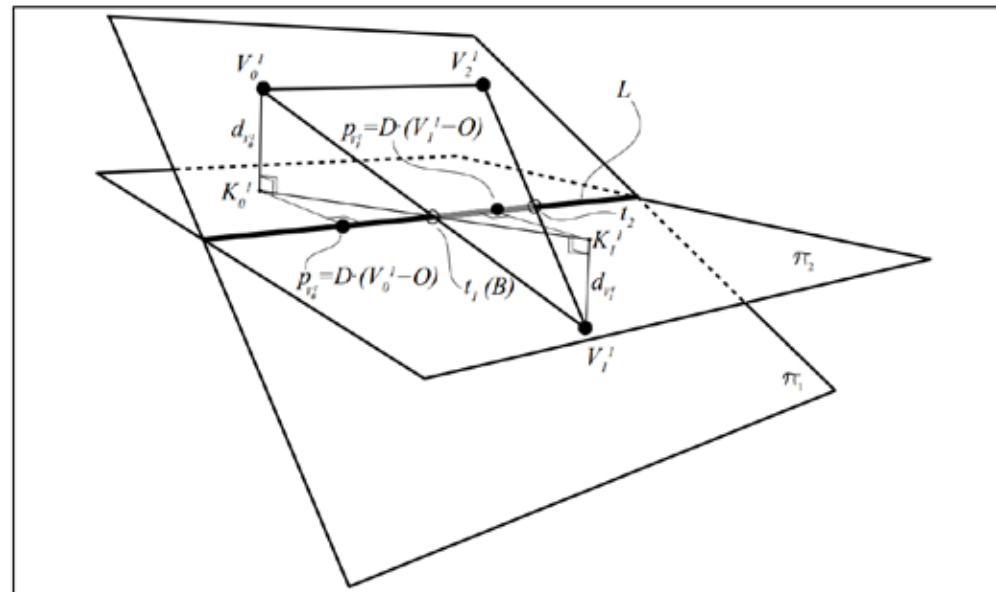
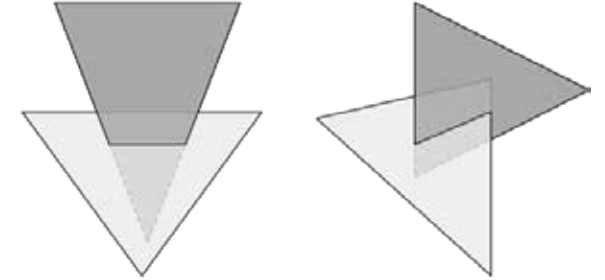
**Algorithm 1:** Test edge-tri intersection for all 6 edges.

$T_1$  intersects  $T_2 \Rightarrow$  Vertices of  $T_1$ ,  $T_2$  straddle plane of  $T_2$ ,  $T_1$  respectively.

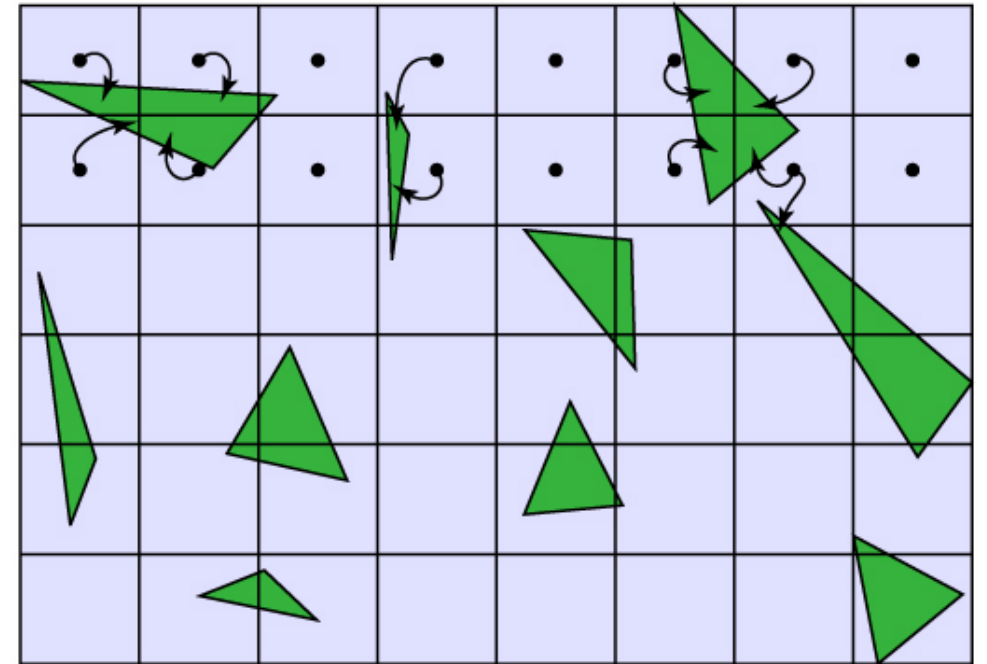
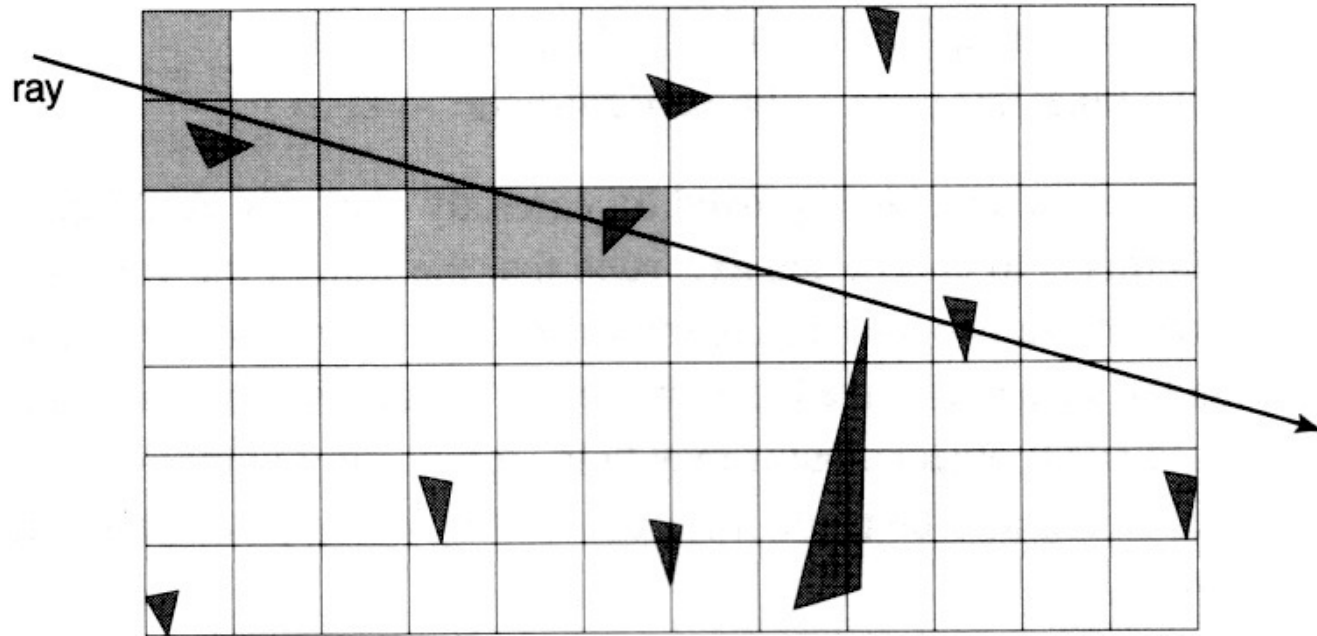
*What if  $T_1$ ,  $T_2$  are co-planar?*

**Algorithm 2:** if (vertices of  $T_1$ ,  $T_2$  on the same side of plane of  $T_2$ ,  $T_1$ ) return false.

The triangles intersect iff the triangle intervals along line of intersection do.



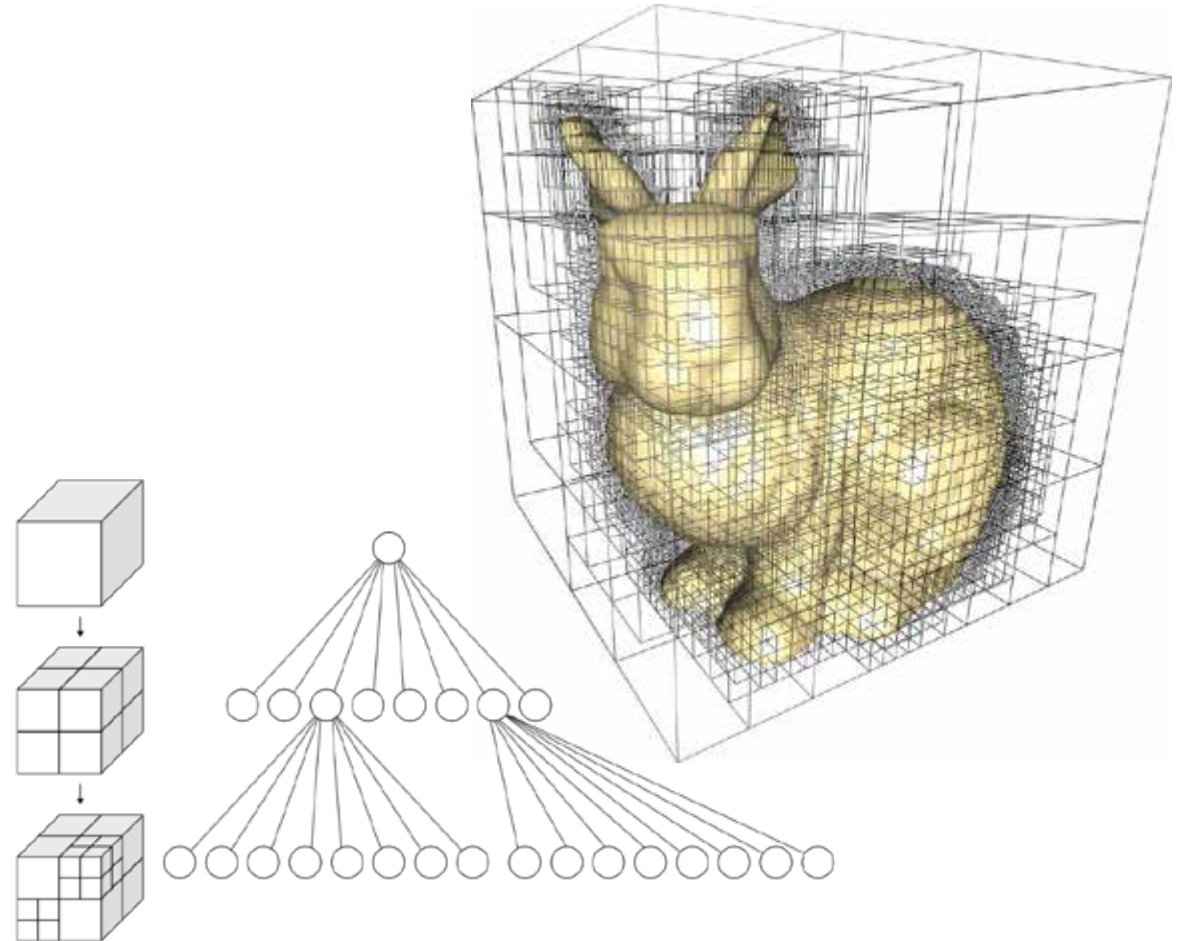
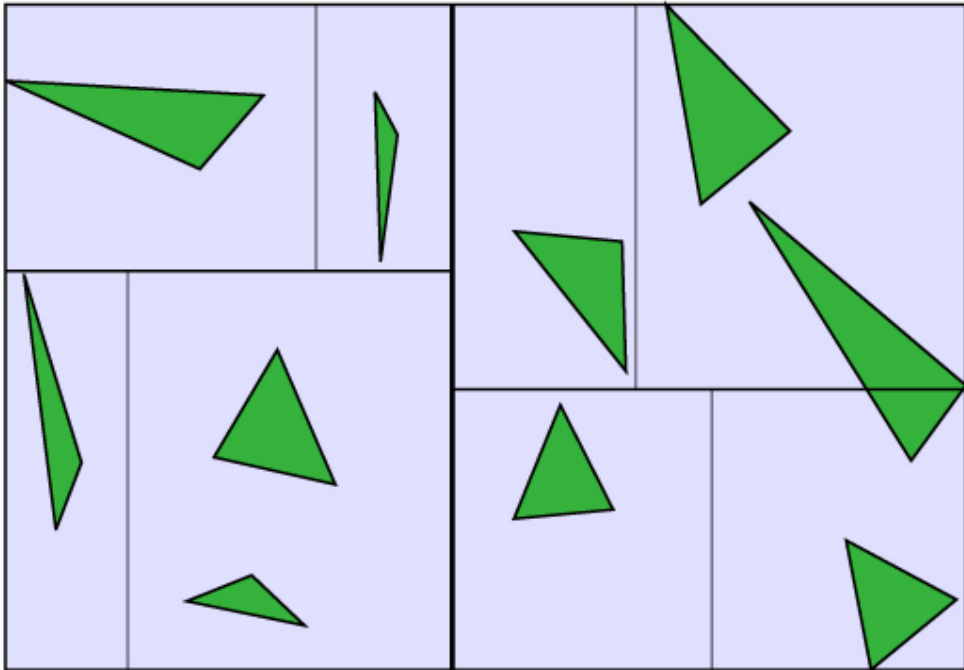
# Regular space subdivision



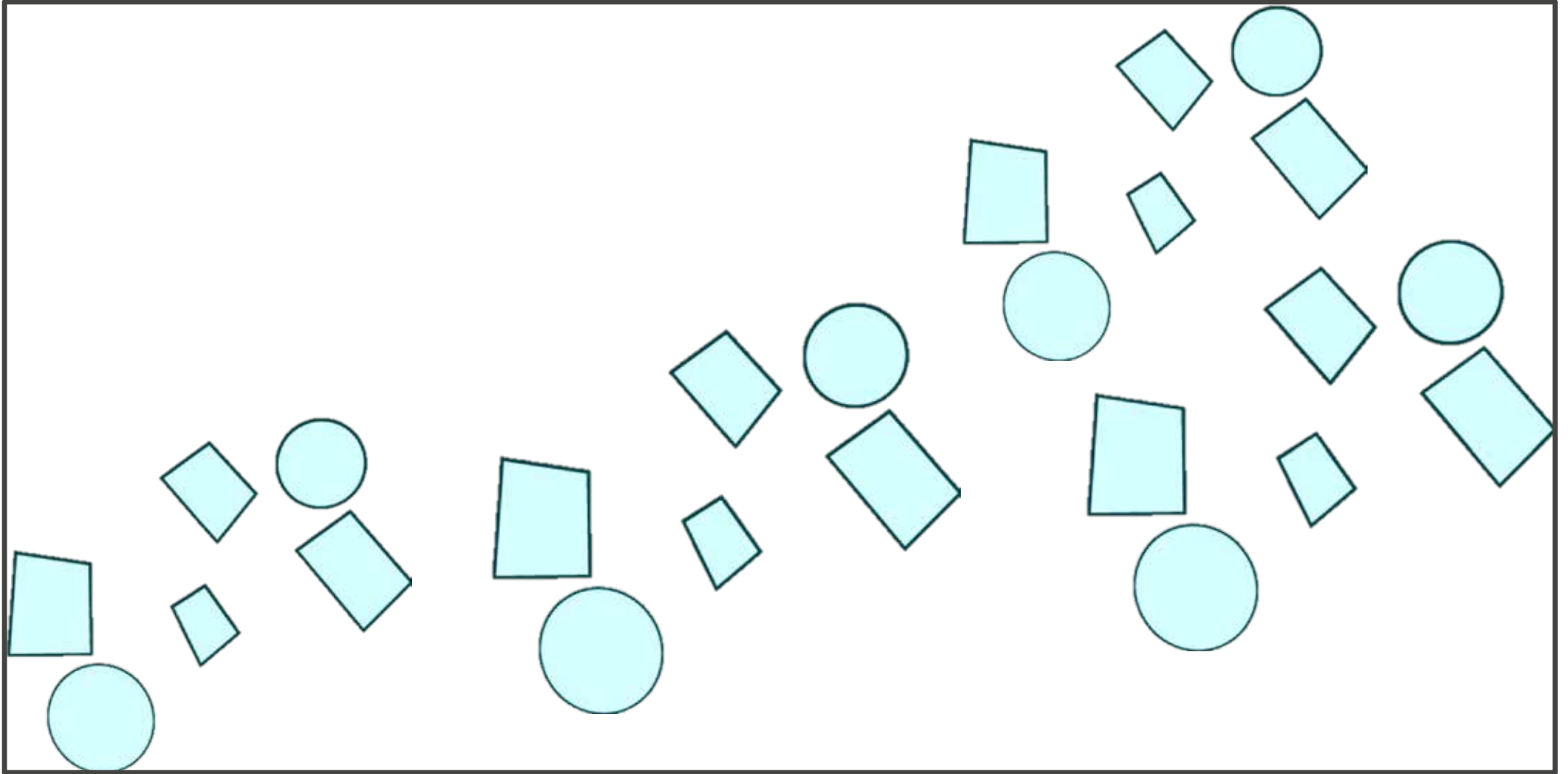
Grid divides space, not objects.

# Non-regular space subdivision

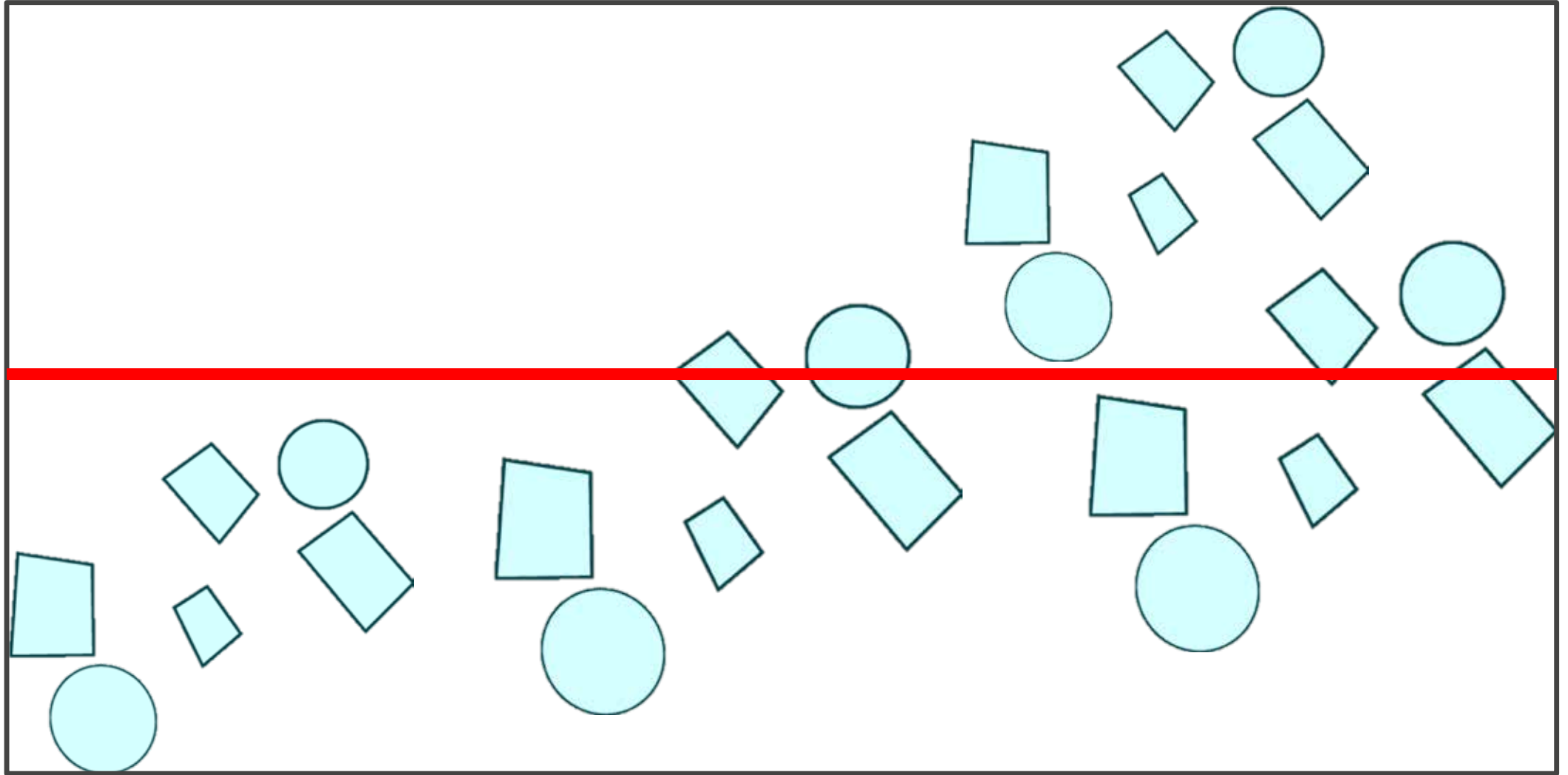
*k*-d Tree  
Octree



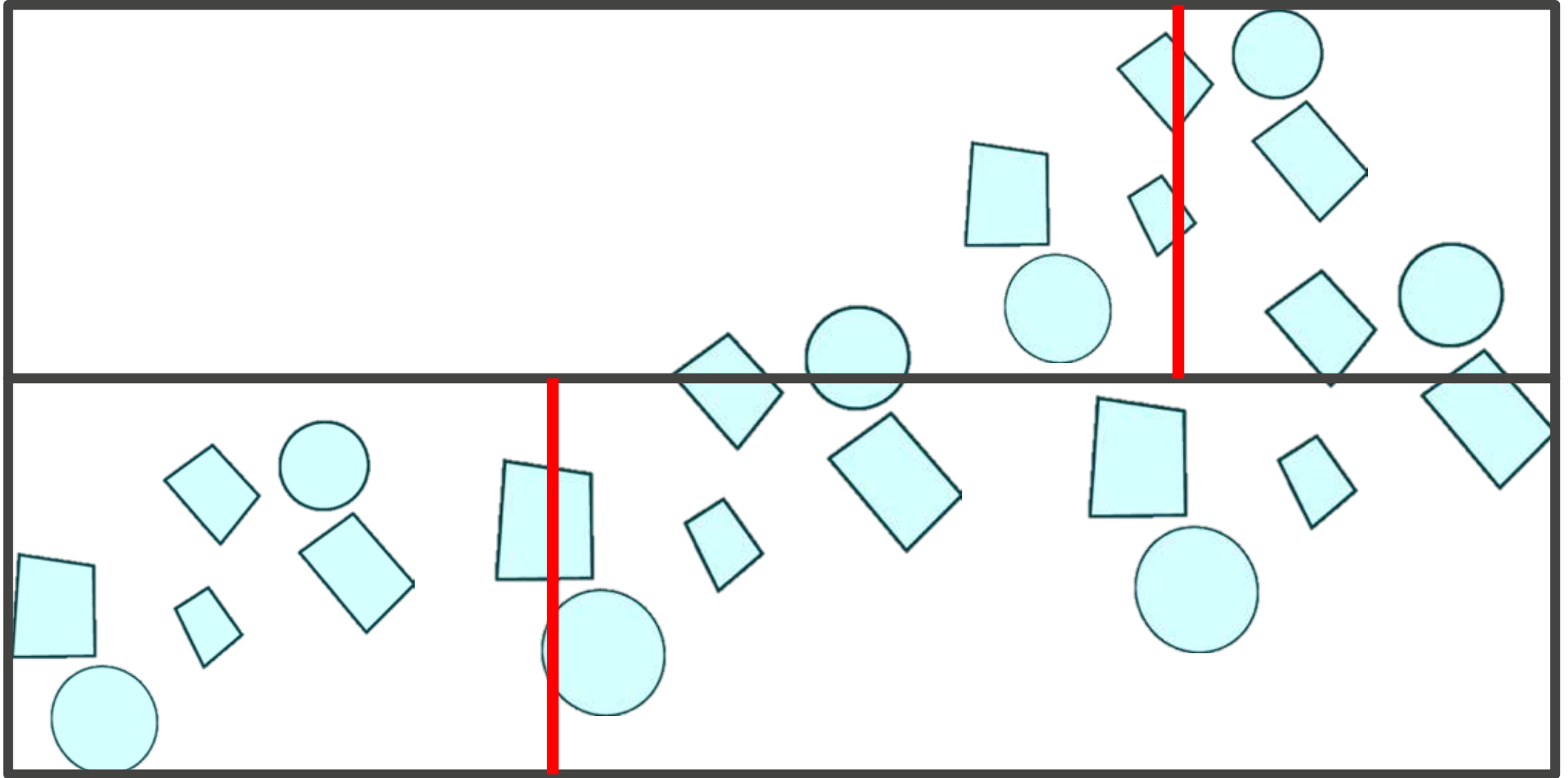
# Constructing a k-d Tree



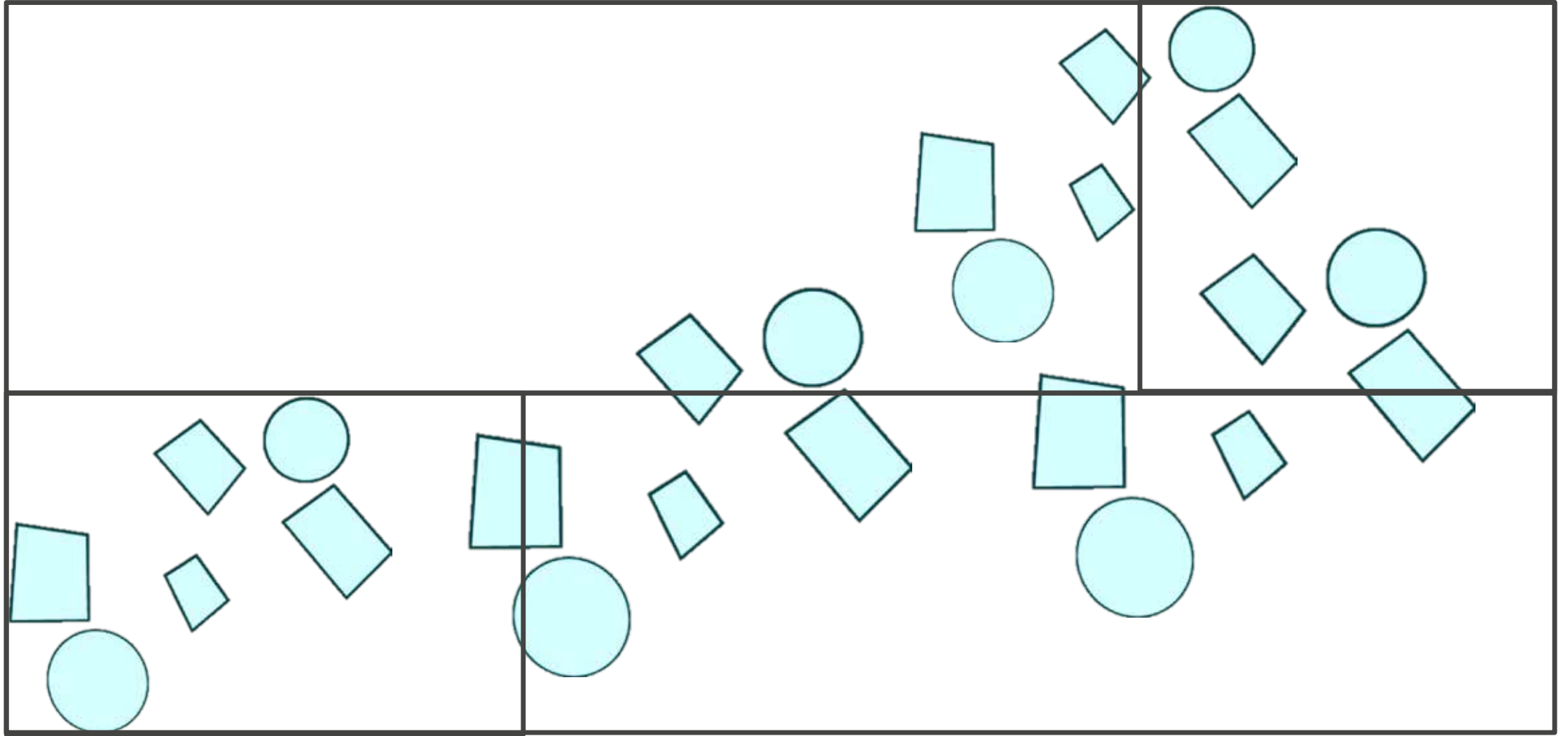
# Constructing a k-d Tree



# Constructing a k-d Tree



# Constructing a k-d Tree



# Assignment #3 Tasks

| File                                                        | What it does                                                                       |
|-------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>insert_box_into_box.py</code>                         | Grow box <code>B</code> so it also encloses box <code>A</code> .                   |
| <code>insert_triangle_into_box.py</code>                    | Grow a box so it encloses a triangle's 3 corners.                                  |
| <code>ray_intersect_box.py</code>                           | Slab test: does a ray hit a <i>solid</i> AABB within <code>[min_t, max_t]</code> ? |
| <code>ray_intersect_triangle.py</code>                      | Marschner–Shirley ray–triangle intersection.                                       |
| <code>ray_intersect_triangle_mesh_brute_force.py</code>     | <code>O(n)</code> closest ray–mesh hit (reference).                                |
| <code>AABBTree.py</code>                                    | Build the hierarchy: split at the midpoint of the longest axis.                    |
| <code>AABBTree_ray_intersect.py</code>                      | Recursive, box-pruned closest ray–object query.                                    |
| <code>nearest_neighbor_brute_force.py</code>                | <code>O(n)</code> nearest point to a query (reference).                            |
| <code>point_box_squared_distance.py</code>                  | Squared distance from a point to a solid box.                                      |
| <code>point_AABBTree_squared_distance.py</code>             | Priority-queue nearest-neighbour query (not recursive!).                           |
| <code>box_box_intersect.py</code>                           | Do two AABBs overlap? (separating-axis test)                                       |
| <code>find_all_intersecting_pairs_using_AABBTrees.py</code> | Broad phase: overlapping leaf-box pairs between two trees.                         |

# Assignment #3 Tasks

Each triangle is coloured by which subtree of the AABB tree it ends up in, at three depths.

